DevOps and Automations TASK – 5

1.Install Terraform on your local machine and write a main.tf file that provisions an AWS EC2 instance with the t2.micro type in the ap-south-1 (Mumbai) region.

Ans :

- Install Terraform
- Download Terraform from the official HashiCorp website:
- Terraform Installation
- After installation, open a new PowerShell/Terminal and verify:
- terraform --version
- You should see something like:
- Terraform v1.x.x
- Create a project folder
- mkdir MusicApp-Mumbai
- cd MusicApp-Mumbai
- Create main.tf
- notepad main.tf
- Paste:
- terraform {
- required_providers {
- aws = {
- source = "hashicorp/aws"
- }
- }
- }

- provider "aws" {
- region = "ap-south-1"

- }

- resource "aws_instance" "music_app_server" {
- ami   = "ami-0f58b397bc5c1f2e8"
- instance_type = "t2.micro"

- tags = {
- Name = "MusicApp-Server"
- }
- }
- Save and close Notepad.
- Initialize Terraform
- terraform init
- Format the file
- terraform fmt
- Validate the configuration
- terraform validate
- Expected:
- Success! The configuration is valid.
- Preview the EC2 instance
- terraform plan
- You should see Terraform planning to create:
- aws_instance.music_app_server
- Create the EC2 instance
- terraform apply
- When prompted:
- Do you want to perform these actions?
- Enter:
- yes

- Verify in AWS
- Open the AWS EC2 Console and select:
- Asia Pacific (Mumbai)
- ap-south-1
- Under Instances, verify:
- Name: MusicApp-Server
- Instance type: t2.micro
- Region: ap-south-1
- State: Running

**2.Modify your Terraform script to add a security group that allows inbound HTTP (port 80) and SSH (port 22) traffic to your EC2 instance.

Hint: Use the aws_security_group resource and reference it in your EC2 instance's vpc_security_group_ids.**

Ans :

- Format:
- terraform fmt
- Validate:
- terraform validate
- Expected:
- Success! The configuration is valid.
- Preview changes:
- terraform plan
- Apply:
- terraform apply
- Enter:
- yes
- Verify in AWS
- Go to AWS Console → EC2 → Instances → MusicApp-Server → Security.

- Your security group should show:
- Inbound rules:

- HTTP TCP 80 0.0.0.0/0
- SSH TCP 22 0.0.0.0/0

3.Create a Terraform variable for the EC2 instance name and update your main.tf to use this variable so you can easily change the instance name without editing the resource block.

Ans :

- Create variables.tf
- In your Terraform project folder, create:
- variables.tf
- Put this inside:
- variable "instance_name" {
- description = "Name of the EC2 instance"
- type = string
- default = "MusicApp-Server"
- }
- Modify main.tf
- Find your EC2 resource:
- resource "aws_instance" "music_app_server" {
- Change the tags section to:
- tags = {
- Name = var.instance_name
- }
- Your EC2 resource should look like:
- resource "aws_instance" "music_app_server" {
- ami = "ami-0f58b397bc5c1f2e8"
- instance_type = "t2.micro"

- vpc_security_group_ids = [aws_security_group.web_ssh.id]

- tags = {
- Name = var.instance_name
- }
- }
- Format and validate
- Run:
- terraform fmt
- terraform validate
- You should get:
- Success! The configuration is valid.
- Change the instance name
- You can now change the name without modifying the EC2 resource.
- For example, create terraform.tfvars:
- instance_name = "My-Playlist-Server"
- Then run:
- terraform plan
- terraform apply
- Enter:
- yes
- The EC2 instance will now have the tag:
- Name = My-Playlist-Server

**4.Write an automation shell script (deploy.sh) that initializes Terraform, applies your configuration, and outputs the public IP of the created EC2 instance.

Hint: Use terraform output to fetch the IP after apply.**

Ans :

- Create deploy.sh
- In your Terraform project folder, create a file named:
- deploy.sh
- Add this script
- #!/bin/bash

- set -e

- echo "Initializing Terraform..."
- terraform init

- echo "Applying Terraform configuration..."
- terraform apply -auto-approve

- echo "Fetching EC2 public IP..."
- terraform output -raw public_ip
- Add the required Terraform output
- Your outputs.tf should contain:
- output "public_ip" {
- description = "Public IP address of the EC2 instance"
- value = aws_instance.music_app_server.public_ip
- }
- Make the script executable
- On Linux/macOS/WSL:
- chmod +x deploy.sh
- Run the deployment script
- ./deploy.sh

- The script will:
- Terraform init
- ↓
- Terraform apply
- ↓
- Get EC2 public IP
- At the end, you should see something similar to:
- Initializing Terraform...
- Applying Terraform configuration...
- Apply complete!

- Fetching EC2 public IP...
- 13.234.xxx.xxx

5.Use ChatGPT to review your main.tf and deploy.sh for best practices and security improvements. Paste your original code and the AI's suggestions, then implement at least one recommended change.

Ans :

- The SSH security group currently allows port 22 from 0.0.0.0/0, which exposes SSH to the entire internet. It should be restricted to a trusted IP address or CIDR range.
- The AMI ID is hardcoded in main.tf. Making the AMI ID a Terraform variable makes the configuration easier to maintain and reuse.
- The deployment script uses terraform apply -auto-approve. This is convenient for automation but can apply changes without a manual confirmation. It should be used carefully, especially in production.

- terraform init could use -input=false in an automated script so the command does not wait for interactive input.
- Terraform state should not be committed to Git. A .gitignore file should exclude .terraform/ and *.tfstate files.
- AWS credentials should never be hardcoded in Terraform files or shell scripts. They should be supplied through environment variables, IAM roles, or a secure CI/CD secret mechanism.